Laboratory Series No. 01

Dr. Rochdi Boudjehem

2025-10-18

Lab 0 — Toolchain Sanity Check and Baseline Performance

This lab ensures your development environment is correctly installed and configured, then collects baseline single-thread measurements you will reuse in later labs (Amdahl/Gustafson, OpenMP, MPI, CUDA).

```
References:
- See course Chapter 1 slides for performance metrics, memory hierarchy, and paradigms con
text.
- Cuda and OpenCL toolchain setup references are not needed now, you can skip them for fir
st labs.
```

---

1) Environment & Setup (verify during the lab)

Choose one option and complete the checks.

- Windows (cmd + MinGW-w64 standalone)
  1. Install MinGW-w64 (WinLibs or niXman). Add `...\mingw64\bin` to PATH.
  2. Open a new cmd.exe and verify:

```
gcc --version
g++ --version
```

  3. MPI on Windows:
     - Recommended: MS-MPI (Runtime + SDK). Compile with MinGW and link `msmpi`, run with `mpiexec`.
     - Alternative: MPICH Windows build if available.
  4. CUDA on Windows: NVCC requires MSVC toolchain. Use the “x64 Native Tools for VS” prompt, or CUDA via WSL/Linux.

- Windows (WSL Ubuntu)
  – Install toolchains:

```
sudo apt update
sudo apt install -y build-essential openmpi-bin libopenmpi-dev ocl-icd-o
pencl-dev nvidia-cuda-toolkit
```

- Linux (native)
  – Ensure GCC, OpenMPI/MPICH, OpenCL ICD, and CUDA Toolkit (if GPU) are installed.

Quick compile/run examples (cmd.exe):

- OpenMP (MinGW-w64 GCC)

```
gcc -O3 -fopenmp lab.c -o lab.exe
lab.exe
```

lab.c is provided in the "Provided Starter Templates" section below.

- MPI (MS-MPI with MinGW-w64 linking)

```
rem Adjust paths if installed elsewhere
set MSMPI_INC=C:\Program Files (x86)\Microsoft SDKs\MPI\Include
set MSMPI_LIB64=C:\Program Files\Microsoft MPI\Lib\x64
gcc -O3 mpi_lab.c -I"%MSMPI_INC%" -L"%MSMPI_LIB64%" -lmsmpi -o mpi_lab.exe
mpiexec -n 4 .\mpi_lab.exe
```

- CUDA (requires MSVC; use Developer Command Prompt or WSL)

```
nvcc -O3 cuda_lab.cu -o cuda_lab.exe
cuda_lab.exe
```

---

2) Tasks to Perform

1. System and timing sanity
   - Write a small C program that prints:
     - CPU brand and core/thread count (any available portable method)
     - Wall-clock timing of a simple loop
   - Hints:
     - Windows: use `QueryPerformanceCounter`/`QueryPerformanceFrequency` or C++ `std::chrono`.
     - Linux/WSL: `clock_gettime` or `std::chrono`.
2. Baseline scalar vector add (single-thread)
   - Implement `for (i=0..N-1) C[i]=A[i]+B[i]` with `N = 100000000` (1e8) floats or doubles.
   - Measure elapsed time. Compute:
     - Throughput GB/s ≈ bytes_moved / time
       - bytes_moved ≈ 3 * N * sizeof(type)
     - Effective FLOP/s (adds per second)
   - Validate a few elements to prevent dead-code elimination.
3. Compiler optimization sweep
   - Build and time the same code with `-O0`, `-O2`, `-O3`.
   - Optional: add `-march=native` on your machine and note impact.
4. Optional (stretch): memory hierarchy experiment
   - Repeat with different `N` and stride to observe cache effects.

---

3) Provided Starter Templates

C (Windows cmd.exe build):

```
// file: lab.c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
```

```c
#include <time.h>
#ifdef _WIN32
#include <windows.h>
static double now_s() {
    LARGE_INTEGER f, c; QueryPerformanceFrequency(&f); QueryPerformanceCounter(&c)
;
    return (double)c.QuadPart / (double)f.QuadPart;
}
#else
#include <sys/time.h>
static double now_s() {
    struct timespec ts; clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}
#endif

int main() {
    const size_t N = 100000000; // 1e8
    float *A = (float*) aligned_alloc(64, N * sizeof(float));
    float *B = (float*) aligned_alloc(64, N * sizeof(float));
    float *C = (float*) aligned_alloc(64, N * sizeof(float));
    if (!A || !B || !C) { fprintf(stderr, "alloc failed\n"); return 1; }
    for (size_t i=0;i<N;i++){ A[i]=1.0f; B[i]=2.0f; }

    double t0 = now_s();
    for (size_t i=0;i<N;i++) C[i]=A[i]+B[i];
    double t1 = now_s();

    // quick check
    printf("C[0]=%f C[N-1]=%f\n", C[0], C[N-1]);

    double dt = t1 - t0;
    double bytes = 3.0 * (double)N * sizeof(float);
    double gbs = bytes / dt / 1e9;
    double flops = (double)N / dt; // 1 add per element
    printf("N=%zu time=%.3f s\n", N, dt);
    printf("Throughput=%.2f GB/s, Adds=%.2f GFLOP/s\n", gbs, flops/1e9);

    free(A); free(B); free(C);
    return 0;
}
```

Build/run (cmd.exe):

```
set CC=gcc
%CC% -O3 -march=native lab.c -o lab.exe
lab.exe
```

Minimal MPI C (MS-MPI linking example):

```c
// file: mpi_lab.c
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv){
    MPI_Init(&argc, &argv);
    int rank, size; MPI_Comm_rank(MPI_COMM_WORLD, &rank); MPI_Comm_size(MPI_COMM_W
```

```
ORLD, &size);
    printf("Hello from rank %d of %d\n", rank, size);
    MPI_Finalize();
    return 0;
}
```

Build/run (cmd.exe):

```
set MSMPI_INC=C:\Program Files (x86)\Microsoft SDKs\MPI\Include
set MSMPI_LIB64=C:\Program Files (x86)\Microsoft SDKs\MPI\Lib\x64
gcc -O3 mpi_lab.c -I"%MSMPI_INC%" -L"%MSMPI_LIB64%" -lmsmpi -o mpi_lab.exe
mpiexec -n 4 .\mpi_lab.exe
```

Note:

```
Adjust paths if MS-MPI is installed elsewhere.
```

---

4) Troubleshooting Tips

- If `gcc` not found in cmd.exe, verify PATH contains your `...\mingw64\bin` and open a fresh cmd.
- If MS-MPI link fails, ensure SDK is installed and `MSMPI_LIB64` path is correct (x64 vs x86). Try quoting paths.
- If runtime `mpiexec` fails, try running `"C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 2 .\mpi_lab.exe`.
- On Windows, prefer `std::chrono` if `clock_gettime` is unavailable.
- Large `N` may exhaust memory on small RAM systems; reduce to 50e6.
- If timings fluctuate, run 3× and report the best and average.

Happy measuring — these baselines will anchor your speedup/efficiency analyses in the next labs.